



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**
título del TFG



Presentado por Nombre del alumno
en Universidad de Burgos — 11 de enero
de 2026

Tutor: nombre tutor



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. nombre tutor, profesor del departamento de nombre departamento, área de nombre área.

Expone:

Que el alumno D. Nombre del alumno, con DNI dni, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado título de TFG.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 11 de enero de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. nombre tutor

D. nombre co-tutor

Resumen

En este primer apartado se hace una **breve** presentación del tema que se aborda en el proyecto.

Descriptores

Palabras separadas por comas que identifiquen el contenido del proyecto Ej: servidor web, buscador de vuelos, android ...

Abstract

A **brief** presentation of the topic addressed in the project.

Keywords

keywords separated by commas.

Índice general

Índice general	iii
Índice de figuras	iv
Índice de tablas	v
1. Introducción	1
1.1. Introducción	1
2. Objetivos del proyecto	3
3. Conceptos teóricos	5
3.1. Secciones	5
3.2. Visión por computadora	5
3.3. Referencias	14
3.4. Imágenes	14
3.5. Listas de items	15
3.6. Tablas	15
4. Técnicas y herramientas	17
5. Aspectos relevantes del desarrollo del proyecto	19
6. Trabajos relacionados	21
7. Conclusiones y Líneas de trabajo futuras	23
Bibliografía	25

Índice de figuras

3.1. Autómata para una expresión vacía	15
--	----

Índice de tablas

3.1. Herramientas y tecnologías utilizadas en cada parte del proyecto	16
---	----

1. Introducción

La monitorización de ecosistemas a partir de imágenes aéreas se ha convertido en una herramienta clave para detectar dinámicas espaciales asociadas a la degradación ambiental y a la pérdida de biodiversidad. En este contexto, las sendas de herbívoros, originadas por el pisoteo continuado y organizadas en redes de trazado complejo, constituyen un indicador relevante de actividad de herbivoría intensa y, por tanto, un potencial marcador de zonas de presión ecológica. No obstante, su identificación manual en ortoimágenes resulta costosa, subjetiva y difícil de escalar, mientras que aproximaciones automáticas clásicas tienden a depender de información topográfica adicional o a producir predicciones a nivel de parche, lo cual limita la delimitación precisa de la estructura y continuidad de las sendas. Frente a ello, la segmentación semántica permite abordar el problema como una clasificación densa a nivel de píxel, incorporando contexto local y global para reconstruir de forma más fiel la geometría de estos patrones sobre imágenes RGB.

Este Trabajo Fin de Grado parte del algoritmo descrito en el artículo base, que evalúa arquitecturas modernas de segmentación semántica para mapear sendas de herbívoros en imágenes aéreas y pone de manifiesto la viabilidad de detectar estas estructuras mediante modelos de aprendizaje profundo. A partir de dicha base, el objetivo principal del proyecto consiste en trasladar esta capacidad de segmentación a un entorno accesible para el usuario final mediante el desarrollo de una aplicación web que permita cargar una imagen, ejecutar el proceso de inferencia y devolver una visualización interpretables, superponiendo y dibujando las sendas detectadas sobre la imagen original. La aplicación incorporará un flujo de uso orientado a la interacción iterativa, de modo que el usuario pueda eliminar la imagen procesada y repetir el procedimiento con nuevas entradas, consolidando así

un prototipo funcional que encapsule el pipeline completo, desde la recepción del dato hasta la presentación del resultado.

Adicionalmente, el trabajo se plantea con una proyección evolutiva hacia un escenario de mayor escalabilidad, donde la fuente de datos no sea únicamente una imagen aislada subida por el usuario, sino servicios cartográficos de alta resolución, como un entorno soportado por Google Earth Engine o por recursos del PNOA. En esa extensión, el usuario seleccionaría una zona de interés sobre el terreno y el sistema obtendría la imagen correspondiente para segmentarla y representar automáticamente la red de sendas en el área escogida, abriendo la puerta a funcionalidades complementarias como la descarga del resultado, el envío por correo electrónico o su integración en flujos de trabajo de análisis territorial.

La memoria se organiza de la siguiente manera: en primer lugar, se presenta el marco teórico necesario para contextualizar la segmentación semántica y los elementos arquitectónicos que la sustentan; a continuación, se describen las arquitecturas consideradas y el enfoque metodológico adoptado para su utilización en el problema de detección de sendas; seguidamente, se detallan el diseño e implementación del sistema, incluyendo el pipeline de procesamiento, la construcción de la interfaz web y la estrategia de despliegue; posteriormente, se exponen las pruebas realizadas y los resultados obtenidos, junto con su análisis; finalmente, se recogen las conclusiones y se definen líneas de trabajo futuro orientadas a la explotación de fuentes geospaciales y a la ampliación de funcionalidades. Como materiales entregables complementarios a la memoria, se incluyen el código fuente del prototipo, los recursos necesarios para su ejecución y una guía de uso y reproducción del entorno, con el fin de facilitar la evaluación, la mantenibilidad y la continuidad del proyecto.

2. Objetivos del proyecto

Este apartado explica de forma precisa y concisa cuales son los objetivos que se persiguen con la realización del proyecto. Se puede distinguir entre los objetivos marcados por los requisitos del software a construir y los objetivos de carácter técnico que plantea a la hora de llevar a la práctica el proyecto.

3. Conceptos teóricos

Algunos conceptos teóricos de L^AT_EX ¹.

3.1. Secciones

Las secciones se incluyen con el comando `section`.

Subsecciones

Además de secciones tenemos subsecciones.

Subsubsecciones

Y subsecciones.

3.2. Visión por computadora

Visión por computadora

La Visión por Computadora, en el contexto de la Inteligencia Artificial, es la disciplina que se enfoca en el diseño y optimización de arquitecturas computacionales para realizar la proyección no lineal de datos de entrada dados por píxeles hacia espacios de características de alta dimensionalidad [7] que encapsulan información semántica estratificada.

¹Créditos a los proyectos de Álvaro López Cantero: Configurador de Presupuestos y Roberto Izquierdo Amo: PLQuiz

El objetivo operacional es la predicción densa a nivel de píxel o la clasificación discriminativa de etiquetas [7, 25], utilizando mapeos jerárquicos de características y garantizando la solvencia de optimización en estructuras ultra-profundas mediante mecanismos residuales [9].

Conceptos teóricos de la Segmentación Semántica

En esta sección se introducen los fundamentos teóricos que sustentan la segmentación semántica moderna en visión por computador, abordándola desde la perspectiva del aprendizaje profundo y de las arquitecturas que permiten realizar predicción densa a nivel de píxel. Se describen los principios que rigen el uso de redes neuronales profundas, haciendo especial énfasis en el papel de las estructuras ultra-profundas y de los mecanismos que facilitan su entrenamiento, como las conexiones residuales y las *skip connections*. Asimismo, se analizan los componentes operativos esenciales de los modelos de codificador-decodificador, incluyendo conceptos clave como *kernels*, *stride*, *downsampling* y *upsampling*, los cuales determinan el compromiso entre resolución espacial y riqueza semántica. Finalmente, se presentan las dos familias arquitectónicas predominantes en este ámbito, las redes neuronales convolucionales y los *Vision Transformers*, destacando sus principios de funcionamiento y su relevancia en tareas de segmentación semántica de alta precisión.

Aprendizaje Profundo

El Aprendizaje Profundo es un paradigma de machine learning basado en el uso de redes neuronales convolucionales (CNN) o arquitecturas alternativas como los *Transformers*, que han surgido como el enfoque dominante para el reconocimiento visual [9, 11]. Funciona de una forma que, una clase de hipótesis de altísima capacidad, realizada por composiciones profundas de operadores paramétricos y no linealidades puntuales, aprende representaciones jerárquicas de los datos optimizando un funcional de riesgo empírico.

El éxito del Aprendizaje Profundo se atribuye a su capacidad para trabajar con arquitecturas muy grandes y profundas, con millones de parámetros, logrando avances significativos en el reconocimiento de imágenes [19, 5, 18, 9]. Este resuelve la necesidad de los sistemas de visión de extraer la representación de características más efectiva a partir de datos complejos [18], automatizando lo que antes requería el diseño manual de características.

Estructuras ultra-profundas

En el marco del Aprendizaje Profundo, la profundidad de las representaciones y de la red es de importancia crucial para muchas tareas de reconocimiento visual [9, 11]. Las arquitecturas modernas han superado significativamente las redes iniciales, con modelos que han rebasado la barrera de las cien capas [11]. Estas estructuras ultra-profundas buscan enriquecer los niveles de características mediante el aumento del número de capas apiladas [9, 11].

Sin embargo, el aumento de la profundidad introduce un desafío conocido como el problema de la degradación. Esta ocurre cuando, al aumentar las capas apiladas, la precisión del entrenamiento se deteriora. Teóricamente, un modelo más profundo no debería tener un error de entrenamiento mayor que su contraparte menos profunda. El hecho de que esto suceda indica que los algoritmos de optimización actuales tienen dificultades para encontrar soluciones que optimicen estas estructuras ultra-profundas [9].

Esto debido a que, a medida que la información o el gradiente pasan por muchas capas, pueden desvanecerse antes de alcanzar el inicio o el final de la red [11].

Predicción Densa, denominada: Dense prediction

La predicción densa es una formulación de inferencia estructurada en la que el sistema estima, para cada ubicación del dominio espacial de la entrada, una variable de salida perteneciente a un espacio de etiquetas o a un espacio continuo [25]. En visión por computador abarca tanto clasificación por elemento, como regresión por elemento, dando lugar a campos de salida con fuerte dependencia local y coherencia geométrica [26, 16].

Desde el punto de vista inductivo, estas tareas se benefician de la *equivarianza traslacional* propia de arquitecturas totalmente convolucionales, del acoplamiento multi-escala para capturar contexto y del control explícito del *output stride* para balancear resolución y campo receptivo [17, 14]. La naturaleza densa impone consideraciones de regularidad de frontera, conservación de detalles de alta frecuencia, manejo del desbalanceo de clases y robustez frente a los problemas introducidos por downsampling [4, 24, 13].

Operativamente, la predicción densa se implementa con *encoders-decoders* convolucionales que combinan *downsampling* para agregar contexto y *up-sampling* para reconstruir resolución, asistidos por *skip connections* por concatenación para preservar información de detalle [19, 2, 1].

Segmentación Semántica

La segmentación semántica es una tarea de predicción densa en Visión por Computadora cuyo objetivo es la clasificación a nivel de píxel [2, 13], mediante la asignación de una etiqueta de clase semántica a cada píxel de una imagen de entrada. A diferencia de tareas como la clasificación, la segmentación semántica proporciona una comprensión completa del escenario, prediciendo así la categoría, ubicación y forma de cada elemento.

Este proceso típicamente se implementa a través de una arquitectura de red neuronal profunda de codificador-decodificador, llamada *encoder-decoder* de manera tradicional [13, 18, 2].

1. **Codificación mediante encoder:** El codificador, a menudo basado en una Red Neuronal Convolutiva (CNN) [7] o un *Vision Transformer (ViT)* [2], toma la imagen de entrada y transforma los datos de esta en una representación de características de alto nivel mediante una reducción de la resolución espacial a través de diversas operaciones como podrían ser el *max-pooling* o el uso de capas convolucionales, obteniendo así como resultado unas capas de características de baja resolución pero semánticamente ricas.
2. **Decodificación por decoder:** El decodificador opera con las características comprimidas del codificador para reconstruir la predicción final. Esto lo hará mediante la utilización de operaciones de sobremuestreo, a menudo implementadas mediante *upsampling* [1, 16] o mediante el uso de índices de *max-pooling* [18, 25], agrandando la entrada para extraer características de alta resolución [18, 13, 1].
3. **Salida y Optimización:** La salida final es un mapa de segmentación que se alimenta a una capa *Softmax* para asignar probabilidades de clasificación a cada píxel de forma independiente [13, 2, 18], generando una predicción densa [18, 2].

Kernel

En el ámbito de las redes neuronales, un *kernel* es un conjunto de filtros con pesos aprendibles que constituyen el componente fundamental de las capas convolucionales para convertir datos de entrada en mapas de características [13]. Operativamente, el *kernel* se desliza sobre la entrada utilizando un *stride*, realizando una operación matemática que produce una salida espacial encargada de capturar jerarquías de información, desde bordes

simples hasta formas complejas [9, 13, 17]. Esta estrategia es intrínseca al procesamiento visual, permitiendo que las computaciones se compartan y que el sistema sea eficiente al manejar imágenes de alta resolución [17].

El diseño del *kernel*, es una decisión arquitectónica crítica que determina el campo receptivo; por ejemplo, mientras que los *kernels* de 3×3 son el estándar por su eficiencia en *hardware*, modelos modernos han demostrado que los *kernels* de 7×7 pueden capturar un contexto global mucho más amplio [17].

Además de la extracción de características, el concepto de *kernel* se extiende a funciones matemáticas de comparación, como el *Central Kernel Alignment (CKA)*, que utiliza un *kernel* lineal para medir la similitud de representación entre los espacios de embebido latente de diferentes arquitecturas [12].

Stride

El *stride* es un parámetro crítico que define la magnitud del desplazamiento de un *kernel* a medida que recorre los datos de entrada en una red neuronal [13, 6]. En las capas del codificador, el uso de un *stride* mayor a uno constituye una técnica de *downsampling* que reduce progresivamente la resolución espacial de los mapas de características [1, 19]. Esta operación permite que los píxeles resultantes abarquen un contexto de imagen de entrada más amplio, facilitando la extracción de características semánticas de alto nivel y mejorando la invarianza traslacional del modelo [13, 1, 17].

En el decodificador, el *stride* es esencial para las operaciones de *upsampling*, donde se utiliza para expandir representaciones comprimidas hacia mapas de alta resolución [13, 1]. El tamaño final de la salida depende de la interacción matemática entre las dimensiones de entrada, el *padding*, el tamaño del *kernel* y el valor del *stride* aplicado [13, 21]. La gestión precisa de este paso es vital para reconstruir con fidelidad la información espacial y asegurar una delineación precisa de los límites de los objetos en tareas de segmentación densa [19, 10, 2].

Transformer

La arquitectura del *Transformer* es un modelo de procesamiento de secuencias basado íntegramente en mecanismos de autoatención, eliminando la necesidad de recurrencia o convoluciones para capturar dependencias [22, 14]. A diferencia de las redes recurrentes que procesan elementos de forma secuencial, el *Transformer* permite el procesamiento en paralelo de toda la

secuencia, lo que facilita el modelado de relaciones de largo alcance y mejora significativamente la escalabilidad en grandes conjuntos de datos [14, 8].

Su estructura estándar se compone de un codificador y un decodificador que apilan múltiples bloques idénticos, cada uno equipado con capas de atención de múltiples cabezales y redes de alimentación *feed-forward* [22, 2].

Mecanismos Residuales

Los mecanismos residuales son esquemas de conexión aditiva que introducen *skip connections* a través de los bloques de una red profunda, de modo que la transformación aprendida se formula como una corrección respecto de la señal que fluye sin distorsión por el atajo [9].

Esta arquitectura estabiliza la propagación hacia delante y hacia atrás, mejora el acondicionamiento efectivo del problema de optimización y habilita redes sustancialmente más profundas; cuando existe desajuste de dimensionalidad, la ruta de identidad se implementa mediante proyecciones, mientras que otras variantes con saltos de identidad estrictos refuerzan el transporte directo de información y gradientes.

En conjunto, los mecanismos residuales facilitan el aprendizaje de funciones de alta complejidad al mantener rutas de baja impedancia para el flujo de señales [20].

Skip Connections

Conexiones encargadas de mitigar la pérdida de información espacial que ocurre durante el *downsampling* en el codificador [2]. Transfieren características de alta resolución desde las capas tempranas del codificador a las capas correspondientes del decodificador, fusionando las características codificadas con los detalles espaciales finos, siendo estos los límites y contornos, del inicio de la red [1, 19].

Para lograr esto, se emplea una capa de concatenación que fusiona la salida de las capas convolucionales del *encoder* con las capas de deconvolución del *decoder* [19, 13].

Upsampling

El *upsampling*, también denominado deconvolución, es la función principal del decodificador, definida como la operación de mapeo que transforma representaciones de características comprimidas de baja resolución en mapas

de características de alta resolución espacial para la clasificación a nivel de píxel [1, 13].

Esto puede implementarse mediante interpolación, convolución transpuesta o decodificadores MLP [13, 4]. La implementación también puede optar por utilizar métodos sin aprendizaje, aunque esto puede resultar en un rendimiento inferior en comparación con los métodos que sí aprenden del entorno [1].

Para preservar los contornos finos y garantizar la localización precisa, el decodificador se basa en la fusión de características de niveles previos del codificador mediante *skip connections* [19, 1, 2].

Downsampling

El *downsampling* es una técnica fundamental utilizada en las redes neuronales convolucionales y en el *encoder* de las arquitecturas de codificador-decodificador para transformar la información de entrada en una representación de características reducida de alto nivel [13, 1].

El propósito central de esta operación es doble: por un lado, aliviar la carga computacional y prevenir el *overfitting*; por otro lado, lograr una mayor *invarianza traslacional* sobre pequeños desplazamientos espaciales en la imagen de entrada, lo cual robustece la clasificación. Esto se realiza a costa de una pérdida progresiva de la resolución espacial en los mapas de características, lo cual reduce el detalle de los límites, un factor que debe compensarse en la fase de decodificación [1, 19, 2].

El mecanismo principal para lograr el submuestreo es la operación de *max-pooling*, que es el núcleo del *downsampling* y se utiliza para disminuir el tamaño del mapa de características [13, 19]. Lo que hace para llevarlo a cabo es dividir dicho mapa de entrada en pequeñas regiones o ventanas. Tras ello, extrae el valor máximo dentro de una de las ventanas, utilizándolo como el único valor representativo para esa área en el nuevo mapa de características reducido. Finalmente, la ventana se desliza a lo largo de la entrada con un *stride* predefinido, permitiendo que cada píxel resultante abarque un contexto de imagen de entrada más amplio a medida que se reduce la resolución [25, 13, 1].

Aunque las sucesivas capas de *max-pooling* y submuestreo pueden lograr una mayor invarianza para una clasificación robusta, la contraparte es una pérdida progresiva de la resolución espacial en los mapas de características. Esta pérdida resulta en una representación cada vez más inexacta en cuanto al detalle de los límites de los objetos, lo cual es perjudicial para tareas

de predicción densa, como la segmentación, donde la delineación precisa es vital [1, 2]. Sin embargo, el *downsampling* tiene un beneficio inherente: logra que cada píxel en el mapa de características posterior abarque un contexto de imagen de entrada más grande.

Red Neuronal Convolutiva (CNN)

Una Red Neuronal Convolutiva es una arquitectura de aprendizaje profundo que se ha convertido en el enfoque dominante para el reconocimiento visual de objetos [11, 9]. Implementa sesgos inductivos inherentes, como la covarianza traslacional, para la extracción eficiente de características jerárquicas, integrando naturalmente características de nivel bajo, medio y alto [17]. En un contexto de predicción densa, como la segmentación semántica, la CNN se extiende típicamente en un marco de codificador-decodificador para realizar una clasificación a nivel de píxel [25, 13, 1].

El funcionamiento en la segmentación se basa en este marco bifásico de codificación y decodificación:

1. Capas Convolutivas, conocidas como: Convolutional Layers

Las capas convolutivas son el núcleo de la CNN y componen la ruta principal del codificador [13, 1]. En estas, se realiza la operación de convolución, que utiliza un conjunto de *kernels* con pesos entrenables para convertir los datos de entrada en mapas de características [13].

El *kernel* se desliza sobre la entrada con un *stride* para producir la salida espacial [9]. Mediante este proceso de *downsampling* codifica la información de entrada en una representación de características de alto nivel downsamplada [13, 1].

2. Capas Deconvolutivas, también conocidas como: Deconvolutional Layers

Las capas deconvolutivas son el mecanismo de *upsampling* más común utilizado en la ruta del decodificador [25, 1]. Mediante el uso de esta técnica, también referida como convolución transpuesta, consiguen mapear los mapas de características comprimidos de baja resolución a mapas de características densos que se aproximan a la resolución de entrada original para la clasificación por elemento [1, 13].

El desafío al que se deben enfrentar estas capas es que la red puede tender a ignorar los detalles debido a la naturaleza de la amplificación, lo que conlleva una salida insatisfactoria [13].

3. Capa de Concatenación, comunmente: Concatenation Layer

La capa de concatenación es un componente de fusión de características utilizado en el decodificador para mejorar la precisión espacial [19, 1, 2]. Para lograr este objetivo, fusiona las características de salida de las capas convolucionales del codificador con las características de las capas deconvolucionales del decodificador.

Dicha fusión se logra a través del uso de *skip connections*, que copian los mapas de características de alta resolución y bajo nivel semántico desde el codificador hacia el decodificador [19, 1, 13]. La concatenación expande estos mapas y, al combinar las características espaciales detalladas con las características semánticas restauradas, asegura una delineación precisa de los límites en la predicción final [19, 1, 2].

Vision Transformer (ViT)

La arquitectura del *Vision Transformer (ViT)* se erige como el paradigma dominante después de las CNN en tareas de reconocimiento visual [17, 14, 2]. Opera al inicio con una fase de tokenización no superpuesta, donde la imagen de entrada es segmentada en parches, aplanados, y proyectados linealmente mediante una transformación a un espacio embebido [2]. A estos *embeddings* se les adjuntan codificaciones posicionales para conservar la información espacial [2].

Esta secuencia de *tokens* enriquecida se introduce en un *encoder* de *Transformer* que consiste en múltiples bloques idénticos, equipados con el mecanismo de *Multi-Head Self-Attention*. El *self-attention* permite que cada *token* capture relaciones de largo alcance y el contexto global con todos los demás *tokens*, facilitando una interpretación más inclusiva de patrones intrincados en los datos [14, 2].

En el contexto de la segmentación densa, el *encoder ViT* finaliza su proceso entregando características codificadas de bajo nivel espacial pero semánticamente ricas. La arquitectura completa luego emplea un decodificador para reconstruir el mapa de segmentación [2]. Este decodificador típicamente incluye capas de *upsampling* para aumentar la resolución espacial de las características comprimidas y utiliza *skip connections* para transferir características de alta resolución desde el *encoder* y así reconstruir el mapa de segmentación con alta fidelidad [2].

La salida final se genera a través de una capa convolucional seguida de una activación *Softmax*, proporcionando un mapa de probabilidades detallado para la clasificación a nivel de píxel [25, 2].

Arquitecturas de Segmentación Semántica

En esta sección se presentan las principales arquitecturas de segmentación semántica empleadas en el ámbito de la visión por computador, las cuales materializan los principios teóricos descritos anteriormente mediante diseños de red específicos orientados a la predicción densa. Se analizan arquitecturas representativas basadas en esquemas de codificador-decodificador y en la agregación jerárquica de características, como U-Net, Feature Pyramid Network y UPerNet, así como enfoques más recientes que incorporan mecanismos de atención global mediante *Transformers*, como SegFormer. Asimismo, se incluye PSPNet, una arquitectura pionera en la integración explícita de contexto multi-escala a través de pirámides de agrupamiento. Cada una de estas arquitecturas introduce estrategias diferenciadas para capturar información semántica y preservar la coherencia espacial, constituyendo referentes fundamentales en el desarrollo y la evolución de los modelos de segmentación semántica.

U-Net

Feature Pyramid Network

UPerNet

SegFormer

PSPNet (Pyramid Scene Parsing Network)

3.3. Referencias

Las referencias se incluyen en el texto usando cite [23]. Para citar webs, artículos o libros [15], si se desean citar más de uno en el mismo lugar [3, 15].

3.4. Imágenes

Se pueden incluir imágenes con los comandos standard de L^AT_EX, pero esta plantilla dispone de comandos propios como por ejemplo el siguiente:



Figura 3.1: Autómata para una expresión vacía

3.5. Listas de items

Existen tres posibilidades:

- primer item.
- segundo item.

1. primer item.
2. segundo item.

Primer item más información sobre el primer item.

Segundo item más información sobre el segundo item.

▪

3.6. Tablas

Igualmente se pueden usar los comandos específicos de $\text{\LaTeX}$ o bien usar alguno de los comandos de la plantilla.

Herramientas	App	AngularJS	API REST	BD	Memoria
HTML5		X			
CSS3		X			
BOOTSTRAP		X			
JavaScript		X			
AngularJS		X			
Bower		X			
PHP			X		
Karma + Jasmine		X			
Slim framework			X		
Idiorm			X		
Composer			X		
JSON		X	X		
PhpStorm		X	X		
MySQL				X	
PhpMyAdmin				X	
Git + BitBucket		X	X	X	X
MikTeX					X
TeXMaker					X
Astah					X
Balsamiq Mockups		X			
VersionOne		X	X	X	X

Tabla 3.1: Herramientas y tecnologías utilizadas en cada parte del proyecto

4. Técnicas y herramientas

Esta parte de la memoria tiene como objetivo presentar las técnicas metodológicas y las herramientas de desarrollo que se han utilizado para llevar a cabo el proyecto. Si se han estudiado diferentes alternativas de metodologías, herramientas, bibliotecas se puede hacer un resumen de los aspectos más destacados de cada alternativa, incluyendo comparativas entre las distintas opciones y una justificación de las elecciones realizadas. No se pretende que este apartado se convierta en un capítulo de un libro dedicado a cada una de las alternativas, sino comentar los aspectos más destacados de cada opción, con un repaso somero a los fundamentos esenciales y referencias bibliográficas para que el lector pueda ampliar su conocimiento sobre el tema.

5. Aspectos relevantes del desarrollo del proyecto

Este apartado pretende recoger los aspectos más interesantes del desarrollo del proyecto, comentados por los autores del mismo. Debe incluir desde la exposición del ciclo de vida utilizado, hasta los detalles de mayor relevancia de las fases de análisis, diseño e implementación. Se busca que no sea una mera operación de copiar y pegar diagramas y extractos del código fuente, sino que realmente se justifiquen los caminos de solución que se han tomado, especialmente aquellos que no sean triviales. Puede ser el lugar más adecuado para documentar los aspectos más interesantes del diseño y de la implementación, con un mayor hincapié en aspectos tales como el tipo de arquitectura elegido, los índices de las tablas de la base de datos, normalización y desnormalización, distribución en ficheros³, reglas de negocio dentro de las bases de datos (EDVHV GH GDWRV DFWLYDV), aspectos de desarrollo relacionados con el WWW... Este apartado, debe convertirse en el resumen de la experiencia práctica del proyecto, y por sí mismo justifica que la memoria se convierta en un documento útil, fuente de referencia para los autores, los tutores y futuros alumnos.

6. Trabajos relacionados

Este apartado sería parecido a un estado del arte de una tesis o tesina. En un trabajo final grado no parece obligada su presencia, aunque se puede dejar a juicio del tutor el incluir un pequeño resumen comentado de los trabajos y proyectos ya realizados en el campo del proyecto en curso.

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo. Éstas pueden ser de diferente índole, dependiendo de la tipología del proyecto, pero normalmente van a estar presentes un conjunto de conclusiones relacionadas con los resultados del proyecto y un conjunto de conclusiones técnicas. Además, resulta muy útil realizar un informe crítico indicando cómo se puede mejorar el proyecto, o cómo se puede continuar trabajando en la línea del proyecto realizado.

Bibliografía

- [1] Vijay Badrinarayanan, Alex Kendall, and Roberto Cipolla. Segnet: A deep convolutional encoder-decoder architecture for image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2016. arXiv:1511.00561.
- [2] Saroj Bala, Kumud Arora, Jeevitha R, Rini Chowdhury, Prashant Kumar, and Shobana Nageswari C. A novel encoder decoder architecture with vision transformer for medical image segmentation. *Journal of Electronics, Electromedical Engineering, and Medical Informatics*, 7(1):176–186, 2025.
- [3] Zachary J Bortolot and Randolph H Wynne. Estimating forest biomass using small footprint lidar data: An individual tree-based approach that incorporates training data. *ISPRS Journal of Photogrammetry and Remote Sensing*, 59(6):342–360, 2005.
- [4] Han Cai, Junyan Li, Muyan Hu, Chuang Gan, and Song Han. Efficientvit: Lightweight multi-scale attention for high-resolution dense prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 17295–17305. IEEE/CVF, 2023.
- [5] José Francisco Díez-Pastora, Francisco Javier González-Moya, Pedro Latorre-Carmona, Francisco Javier Pérez-Barbería, Ludmila Kuncheva, Antonio Canepa-Oneto, Álar Arnaiz-González, and César García-Osorio. Remote sensing colour image semantic segmentation of large herbivorous mammals trails. *Remote Sensing (submitted manuscript)*, 2025. Manuscript compiled May 19, 2025. Author’s Original Manuscript (AOM) available in arXiv.

- [6] Vincent Dumoulin and Francesco Visin. A guide to convolution arithmetic for deep learning, 2018.
- [7] Emergent Mind. Vision encoder: Architectures, training, efficiency, and applications. Technical report, 2025. Unpublished manuscript.
- [8] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2024.
- [9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778. IEEE, 2016.
- [10] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, et al. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019. arXiv:1905.02244.
- [11] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4700–4708, 2017.
- [12] Andreea Iana, Goran Glavaš, and Heiko Paulheim. Peeling back the layers: An in-depth evaluation of encoder architectures in neural news recommenders. In *Proceedings of INRA 2024: 12th International Workshop on News Recommendation*, pages 1–10. ACM, 2024. arXiv:2410.01470.
- [13] Ankang Ji, Alvin Wei Ze Chew, Xiaolong Xue, and Limao Zhang. An encoder-decoder deep learning method for multi-class object segmentation from 3d tunnel point clouds. *Automation in Construction*, 137:104187, 2022.
- [14] Salman Khan, Muzammal Naseer, Munawar Hayat, Syed Waqas Zamir, Fahad Shahbaz Khan, and Mubarak Shah. Transformers in vision: A survey. *ACM Computing Surveys*, 54(10s):1–41, 2022.
- [15] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.
- [16] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 936–944. IEEE, 2017.

- [17] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11976–11988. IEEE/CVF, 2022.
- [18] Asif Ahmed Nelay and Maxime Turgeon. A comprehensive study of auto-encoders for anomaly detection: Efficiency and trade-offs. *Machine Learning with Applications*, 17:100572, 2024.
- [19] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Proceedings of the International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241. Springer, 2015.
- [20] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. In *Proceedings of the Deep Learning Workshop at the 32nd International Conference on Machine Learning (ICML)*. arXiv preprint arXiv:1505.00387, 2015.
- [21] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 6105–6114. PMLR, 2019.
- [22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.
- [23] Wikipedia. Latex — wikipedia, la enciclopedia libre. <https://es.wikipedia.org/w/index.php?title=LaTeX&oldid=84209252>, 2015. [Internet; descargado 30-septiembre-2015].
- [24] Tete Xiao, Yingcheng Liu, Bolei Zhou, Yuning Jiang, and Jian Sun. Unified perceptual parsing for scene understanding. In *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, 2018.
- [25] et al. Zhang. Paper published in automation in construction. *Automation in Construction*, 2022. Elsevier article extracted from PDF 1-s2.0-S0926580522000607.
- [26] Hengshuang Zhao, Jianping Shi, Xiaojuan Qi, Xiaogang Wang, and Jiaya Jia. Pyramid scene parsing network. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2881–2890. IEEE, 2017.